

Using Generative AI in Higher Programming Education: An Empirical Evaluation

Dennis Grewe¹^a, Raluca-Maria Vedislav¹ and W. Daniel Scherz²^b

¹Department of Computer Science and Engineering, Esslingen University of Applied Sciences, Esslingen, Germany

²Department of Computer Science, Konstanz University of Applied Sciences, Konstanz, Germany

Keywords: Generative AI, Programming Education, Best Practices, Educational Technology, ChatGPT, Scaffolding.

Abstract: The growing use of generative artificial intelligence (GenAI) tools like ChatGPT presents both opportunities and challenges for programming education, especially for novices. While these tools can assist with code generation, debugging and conceptual understanding, there is a risk that they will promote superficial learning and the uncritical adoption of generated code. This paper presents the development and empirical evaluation of a best-practice guideline designed to support students in the reflective and effective use of generative AI in programming contexts. We conducted an experimental study with 36 first-semester computer science students, comparing a control group using AI without guidance against an experimental group equipped with our structured guide. The results show that while the quantitative performance metrics remained similar between the groups, the students using the guideline demonstrated a significantly higher self-reported understanding of their solutions and exhibited more reflective AI interaction patterns. Observational data revealed that guided users critically evaluated (66.7% vs 55.6%), modified (61.1% vs 22.2%), and rejected (38.9% vs 16.7%) AI-generated suggestions more frequently. A supplementary AI guidance workshop conducted provided convergent validation: students independently identified debugging and conceptual understanding as their primary skill gaps requiring AI support.

1 INTRODUCTION

The emergence of generative AI (GenAI) tools such as ChatGPT, Claude, and Gemini is transforming learning and education by democratizing access to information and reshaping how people work and think.

Students increasingly rely on AI as an academic companion, and adoption has been rapid: Garrel et al. (Garrel et al., 2025) report that over 90% of students in Germany now use AI for academic purposes, up from 63% in 2023. Our own baseline survey of first-semester computer science students (n=114) reflects this, with 98.2% having already used AI tools at the start of their studies.

Although GenAI presents significant opportunities for personalized learning and cognitive support (e.g., (Huang et al., 2025)), it also introduces substantial risks. Students may develop an over-reliance on AI systems, engage in uncritical copy-and-paste behaviour, or fail to develop fundamental problem-

solving skills (Prather et al., 2024). The challenge lies not in the availability of these tools, but in ensuring their pedagogical integration into programming education.

These concerns are especially pressing in programming education, where the task demands abstract thinking, practical application of theory, and mastering the syntax of a programming language suitable for the problem (Lahtinen et al., 2005). Tools such as GitHub Copilot, Cursor AI, Claude Code, and Google Gemini can provide valuable scaffolding, debugging support, and adaptive explanations. But without proper guidance, they risk undermining the deep learning and critical thinking essential to becoming a proficient programmer (Liu et al., 2024).

Recent studies have explored instructional strategies such as integrating AI tools into lectures (Liu et al., 2024) or raising student awareness of GenAI limitations (Yilmaz and Karaoglan Yilmaz, 2023), yet a comprehensive, empirically evaluated set of best-practice guidelines for reflective GenAI use in programming education remains absent. This paper addresses that gap by presenting the development and

^a <https://orcid.org/0000-0002-9318-5424>

^b <https://orcid.org/0000-0002-5435-2349>

empirical evaluation of such a guideline, investigating two research questions:

1. How can students be effectively supported in their use of GenAI tools for programming tasks?
2. Can a structured guideline improve the quality and effectiveness of GenAI interactions to achieve better learning outcomes?

We conducted an initial experiment with 36 first-semester computer science students completing a time-boxed programming challenge, comparing a control group using GenAI without guidance against an experimental group equipped with the guideline. A supplementary AI guidance workshop with a subsequent cohort (n=80) provided further validation of key patterns and additional evidence on student needs and guideline reception.

The remainder of the paper is structured as follows: Section 2 reviews related work; Section 3 describes the guideline design and experimental setup; Section 4 presents the findings; Section 5 discusses them; and Section 6 concludes with open questions and future work.

2 BACKGROUND AND RELATED WORK

Before exploring the use of generative AI in programming education in greater depth, the following section reviews the challenges of programming education, how AI tools have been applied to address them, and the associated risks of GenAI integration.

2.1 Challenges in Programming Education

Learning a new programming language presents unique challenges, particularly for novice learners. Research identifies several key areas where students struggle (cf. (Duran et al., 2018; Lahtinen et al., 2005; Zhou et al., 2021)):

- cognitive overload from simultaneous processing of syntax, semantics, and problem-solving strategies,
- difficulties with abstract thinking required for concepts like memory management and control flow,
- frustration with syntax errors and cryptic error messages,
- and gaps between theoretical understanding and practical application

Cognitive Load Theory provides a framework for understanding these challenges, distinguishing between intrinsic load (inherent task complexity), extraneous load (irrelevant information), and relevant load (beneficial cognitive processing) (Salleh et al., 2018). Effective educational interventions aim to reduce the cognitive load while promoting relevant processing through scaffolding techniques. Such techniques may include breaking complex problems into smaller sub-tasks, providing pre-defined working examples, offering contextual hints or feedback, and gradually fading support as learners gain proficiency.

2.2 AI Support in Programming Education

Generative AI tools show considerable potential to solve classic programming education challenges. Tian et al. () demonstrated that ChatGPT was able to successfully solve all simple programming tasks and approximately 65% of more complex problems on LeetCode – an online platform widely used by students and professionals to practice coding and prepare for technical interviews. In many cases, the AI-generated solutions were not only correct but also more efficient than those written by humans.

The popular Harvard CS50 computer science beginners course provides a notable example of structured AI integration (Liu et al., 2024). Their custom GPT-4-based assistant provided scaffolded support through didactic hints rather than direct solutions, implemented safety mechanisms against prompt injection, and used Retrieval-Augmented Generation to improve response reliability. Students reported positive experiences with the always-available, patient interaction model.

Huang et al. (Huang et al., 2025) demonstrated that AI-based scaffolding through a CodeFlow Assistant significantly improved programming performance and reduced cognitive load for novice learners. Their system provided structured support through flowcharts, cloze coding exercises (fill-in-the-blank exercises), basic solutions, and advanced optimization examples, leading to significantly better programming outcomes.

2.3 Risks and Challenges of Using AI in Programming Education

While generative AI holds clear potential for programming education, it raises important concerns about student dependence, academic integrity, and problem-solving development. Research on critical thinking remains inconclusive: some learners report

that AI encourages deeper analytical thinking, while others note a decline in independent reasoning (Yilmaz and Karaoglan Yilmaz, 2023). Grassini (Grassini, 2023) highlights further risks, including hallucinations, lack of conceptual understanding, and training data bias, which are particularly problematic where accuracy is essential. Prather et al. (Prather et al., 2024) found that novice programmers with longer task completion times exhibited more meta-cognitive difficulties and higher AI suggestion acceptance rates, with many unconsciously adopting AI-generated code while believing they had worked independently – an “illusion of progress” now commonly known as *vibe coding*.

These findings underscore both the opportunities and risks of GenAI in programming education. Fostering reflective, responsible AI engagement appears to be among the most effective strategies for mitigating over-reliance, integrity concerns, and conceptual misunderstandings.

3 METHODOLOGY AND GUIDELINE DEVELOPMENT

The following section presents the development of a best-practice guideline for integrating AI into programming education and the methodology used to empirically evaluate its effectiveness in a real-world programming challenge.

3.1 Best-Practice Guideline Design

The guideline¹ follows a four-phase instructional design (introduction and goal setting; problem identification; strategy provision; reflection and transfer), aligned with Merrill’s First Principles of Instruction (Merrill, 2002), and addresses six core areas:

Foundation and Motivation: AI capabilities and limitations; the typical learning curve from euphoria through disillusionment to productive collaboration.

AI Technology Overview: LLM types (GPT, Claude, Copilot) and their respective strengths and application areas.

Programming-Specific Guidelines: Critical evaluation of outputs, understanding hallucinations, and maintaining responsibility for code quality.

Prompt Engineering: Prompt components (instruction, context, input, output indicators) and techniques (zero-shot, few-shot, chain-of-thought).

Best Practices: Language choice, context provision, constraint definition, and session management.

¹<https://github.com/HSE-DNS/ai-programming-guide>

Reflection: Examples and strategies to evaluate AI interactions and develop meta-cognitive awareness.

Each area is accompanied by positive and negative examples to raise awareness of associated risks and opportunities for programming novices.

3.2 Experimental Design

To evaluate the effects of the developed guideline, a controlled experiment was conducted with first-semester computer science students enrolled in the “Programming in C” course at our university across multiple computer science-related degree programs in May 2025 during the summer semester. The course introduces fundamental programming concepts using the C programming language and is accompanied by practical programming assignments, with weekly submissions, that are aligned with the lecture topics. Successful completion of assignments counts toward course credit.

The experiment was integrated into the regular practical exercise of the course. Students were invited to voluntarily participate in an AI programming experiment. Successful participation counted as completion of that week’s assignment. Those who refused to participate were required to submit the standard weekly exercise instead. Out of a potential 70 students, 36 ultimately volunteered to take part in the experiment.

3.2.1 Group Formation and Preparation

Participants were randomly assigned to two equal groups: Group A (control, n=18) used AI without any additional guidance; Group B (experimental, n=18) received the best-practice guideline as part of a 20-minute presentation covering all aspects of the guideline and how to interact with an AI tool. Both groups completed an identical programming task to implement a simplified Snake Game in the C programming language.

3.2.2 Computer & Programming Setup

During the experiment, the computer environment consisted of a programming editor and compiler infrastructure (here: Visual Studio Code with gcc on Linux) in the university computer labs. As the AI tool, students were only allowed to use *bwGPT* (KIT, Online) – a state-run AI chatbot service rolled out across public universities in Baden-Wuerttemberg, Germany. It is a privacy-compliant, campus-wide AI infrastructure for teaching and learning based on OpenAI’s GPT-4 and GPT-4o models.

3.2.3 Experiment and Data Collection

During the experiment, different data collection assessments took place. First, a pre-test questionnaire was conducted, including 29 questions assessing demographics, programming experience, AI usage patterns, and conceptual knowledge as part of a multiple-choice test. Afterwards, each participant completed the task with optional (Group A) or guided (Group B) AI support. During the programming task, structured behavioural observations were carried out by four volunteering senior students (final-year CS undergraduates), each assigned to one group and thus not blinded to condition. Behaviour was recorded using a standardized template with four predefined categories (critical evaluation, code modification, output rejection, direct adoption); inter-rater reliability was not computed. Observers were not permitted to answer participant questions. After finishing the programming tasks, a post-test questionnaire was conducted (12-15 questions) evaluating subjective experiences, AI usage patterns, and guideline effectiveness. Finally, each student submitted their code and the chat history between the student and the AI – a special feature of bwGPT to learn about the interactions.

3.2.4 Follow-Up Validation: AI Guidance Workshop

To validate the findings from the controlled experiment and assess real-world applicability, a follow-up AI guidance workshop was conducted in the subsequent semester as part of the regular laboratory classes of the introductory programming course with first-semester students from the same degree programs ($n=80$). The training was offered as a voluntary session, mirroring the voluntary participation model of the original programming experiment.

The workshop presented the complete best-practice guideline in a hands-on format, covering all six core areas described above. Participants completed pre- and post-training questionnaires assessing AI usage patterns, programming skill confidence, and guideline reception.

Demographics confirmed a representative sample: 95% were first-semester students, 94% were aged 18–25, distributed across three different majors, namely IT Security (47.6%), Computer Science (25.6%), and Software Engineering (26.8%) programs. This cohort represents digital natives at the beginning of their programming education journey.

4 RESULTS

This section presents the findings of the experiment presented in Section 3. After collecting all relevant data, a structured analysis was conducted focusing on four key areas. First, the baseline characteristics of the participants are outlined to ensure group comparability. Then, the results of students' subjective perceptions from the post-test questionnaire are presented. Third, the quality of interaction with generative AI tools during the programming task is explored. Finally, the actual task performance and reception of the best-practice guideline are presented.

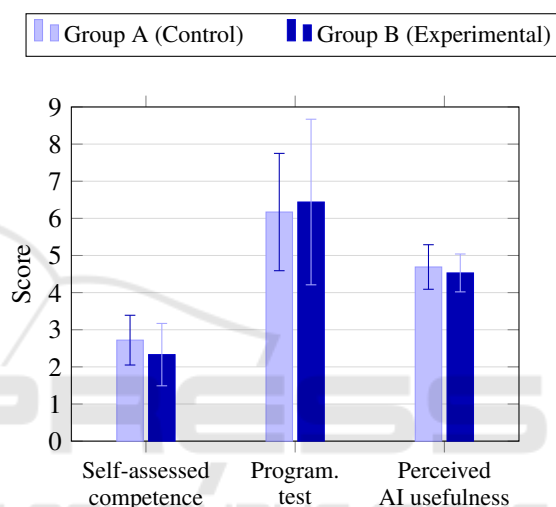


Figure 1: Pre-test results comparing Group A (control) and Group B (experimental). Self-assessed programming competence (1), Programming competence test (2) and Perceived usefulness of AI tools (3).

4.1 Baseline Characteristics

The pre-test questionnaire was used to assess participants' demographics, prior experience with programming and AI tools, and initial self-assessments. It confirmed comparable baseline conditions between groups.

Participants averaged 25.0 years of age ($SD = 8.9$), with no statistically significant differences in age, programming experience, or prior AI usage. Self-assessed programming competence (5-point Likert scale) was similar across groups (Group A: $M = 2.72$, $SD = 0.7$; Group B: $M = 2.33$, $SD = 0.8$; $t = 1.54$, $p > 0.05$), as were scores on a nine-point competence test (Group A: $M = 6.17$, $SD = 1.6$; Group B: $M = 6.44$, $SD = 2.2$; $t = -0.43$, $p > 0.05$). Prior AI tool usage was near-universal (91.7%), with ChatGPT ($n = 29$), GitHub Copilot ($n = 11$), and Google Gemini ($n = 11$)

the most commonly used. Perceived usefulness of AI tools was consistently high in both groups (Group A: $M = 4.69$, $SD = 0.7$; Group B: $M = 4.53$, $SD = 0.6$), with no significant difference (cf. Figure 1).

4.2 Task Performance and Understanding

After the programming task, participants completed a post-test questionnaire assessing subjective task difficulty, perceived solution quality, and understanding of their final code. Perceived difficulty was slightly lower in Group B ($M = 3.89$, $SD = 0.9$ vs. $M = 4.00$, $SD = 0.8$, $p = 0.693$), and no significant difference emerged in self-assessed solution quality. However, Group B reported significantly higher understanding of their solutions ($M = 3.61$ vs. $M = 2.89$, $p = 0.048$), indicating a potential benefit of structured AI guidance on cognitive integration.

All participants used AI during the task (100%, $n = 36$). Usage patterns differed across categories: code generation (A: 13, B: 11), conceptual understanding (A: 7, B: 9), debugging (A: 4, B: 8), and syntax lookup (A: 6, B: 1). Group B thus applied AI more toward debugging and conceptual understanding rather than syntax lookup (cf. Figure 2).

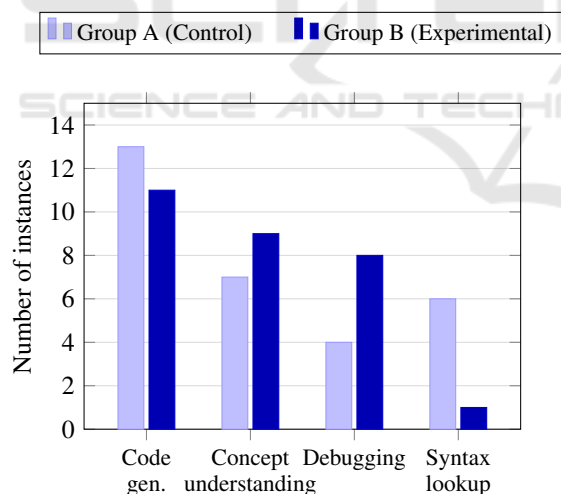


Figure 2: Post-test results: Usage categories of AI during the experiment. Code generation (1), Conceptual understanding (2), Debugging (3), and Syntax lookup (4).

4.3 AI Interaction Quality

To assess AI interaction quality, structured behavioural observations were conducted by senior students alongside code similarity analysis. Although AI usage frequency was similar across groups (Group A: $M = 3.00$, $SD = 1.3$; Group B: $M = 2.94$, SD

$= 1.1$), interaction quality differed notably: Group B participants showed higher rates of critical evaluation (66.7% vs. 55.6%), modification (61.1% vs. 22.2%), and rejection (38.9% vs. 16.7%) of AI-generated outputs, suggesting more reflective engagement (Figure 3). Self-reported confidence in AI interaction was also higher in Group B ($M = 2.71$ vs. $M = 2.27$ on a 5-point Likert scale). Code similarity analysis, using token-level string matching against AI-exported reference code from the chat history, further indicated greater adaptation in Group B ($M = 45.37\%$) compared to Group A ($M = 64.90\%$), though this difference was not statistically significant ($t = 0.99$, $p > 0.05$), and direct adoption of AI-generated code occurred at similar frequencies across both groups (cf. Figure 3).

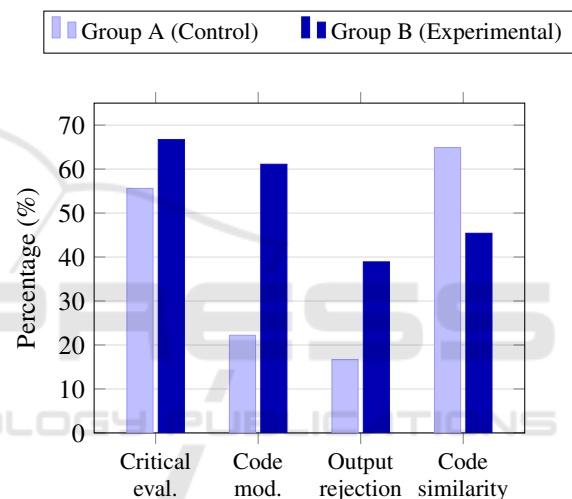


Figure 3: Behavioural observations: Quality of AI interactions. Critical evaluation (1), Code modification (2), Output rejection (3), Code similarity (4).

4.4 Guideline Reception and Usage

Group B participants received and applied the best-practice guideline, with post-test questionnaires (5-point Likert scales) indicating moderate engagement: active usage ($M = 2.83$, $SD = 1.0$), perceived helpfulness ($M = 2.89$, $SD = 1.0$), and intention to reuse in future tasks ($M = 3.44$, $SD = 0.9$). These values suggest the guideline was found somewhat useful, though limited familiarization time likely constrained deeper adoption. Notably, a statistically significant positive correlation was found between prior AI experience and perceived task usefulness ($r = 0.37$, $p = 0.035$), suggesting that students already familiar with AI tools were better positioned to leverage them during the task.

4.5 Supplementary Validation: AI Guidance Workshop

The AI guidance workshop, conducted as part of regular laboratory classes, provided an opportunity to validate key findings from the controlled experiment in a broader educational context. Although direct statistical comparison between cohorts is limited due to differing sample sizes and experimental conditions, several convergent patterns emerged that strengthen the interpretation of our primary results.

4.5.1 AI Usage Patterns

Pre-training assessment confirmed high AI integration among first-semester students. Of 80 participants, 55% reported daily AI chatbot usage, with an additional 21.2% using AI frequently (more than 10 times). Only 5% had never used AI chatbots. For programming-specific tasks, 81.2% had used AI multiple times or regularly, with only 7.5% reporting no prior AI use for coding.

These findings align with the 91.7% prior AI usage rate observed in the programming experiment, suggesting that high AI adoption is consistent across cohorts and semesters.

4.5.2 Skill Confidence and AI Use Cases

Students rated their confidence across programming skills on a 5-point scale. Debugging emerged as the weakest skill area. Only 30% of students felt confident or very confident, which directly mirrors the primary experiment results, where debugging was the most frequent AI use case ($\approx 20.3\%$).

Post-training, when asked which topics warranted additional practice, debugging with AI emerged as the most requested topic (31.8%), followed by prompting strategies (22.7%), test writing (22.7%), and understanding concepts with AI (22.7%).

This convergence – debugging as both the primary AI use case (programming experiment) and the greatest skill gap requiring AI support – validates the guideline’s emphasis on critical evaluation and independent problem-solving skills.

4.5.3 Workshop Reception

Participants evaluated the workshop format and pacing: 63% rated the pacing as “just right,” while 17.4% found it too slow, 17.4% found it uneven, and only 2.2% considered it too fast. This feedback suggests that the guideline content can be effectively delivered within typical instructional time frames.

The 59% post-training survey completion rate (47 of 80) represents typical voluntary survey attrition in educational settings but limits longitudinal analysis. Nevertheless, the consistent patterns between the controlled experiment and the workshop support the generalizability of our primary findings. Figures 4 and 5 present key metrics from the AI guidance workshop session.

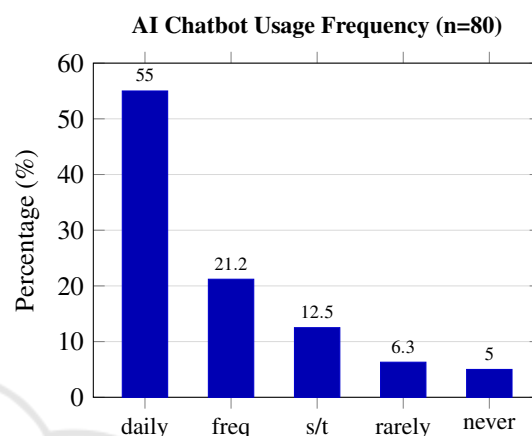


Figure 4: AI chatbot usage frequency among workshop participants (n=80). 76% report frequent or daily usage.

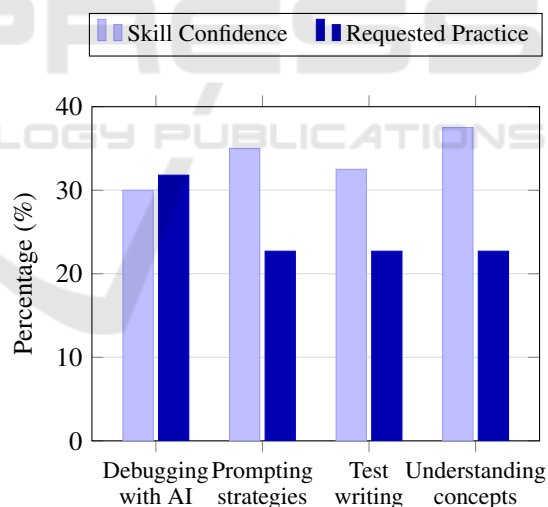


Figure 5: Comparison of skill confidence levels with requested practice topics. Debugging shows lowest confidence and highest demand for additional practice.

5 DISCUSSION

This section interprets the observed patterns of AI interaction and learning outcomes in the context of existing literature, before addressing pedagogical impli-

cations, key limitations, and directions for future research.

5.1 Enhanced Learning through Reflective AI Use

Our findings demonstrate that structured guidance fosters more reflective and deliberate AI interaction, even without immediate performance improvements. The statistically significant difference in self-reported solution understanding ($p = 0.048$) suggests the guideline primarily affected cognitive engagement rather than task completion efficiency, aligning with Prather et al.'s (Prather et al., 2024) distinction between superficial code production and deeper conceptual understanding. Although both groups delivered functionally comparable code, Group B participants engaged more critically with AI-generated suggestions — evaluating, modifying, or selectively rejecting them — indicative of active knowledge construction rather than passive reliance. Observational data support this: Group B demonstrated higher critical assessment (66.7% vs. 55.6%) and adaptive modification of AI suggestions (61.1% vs. 22.2%), consistent with scaffolding principles described by Huang et al. (Huang et al., 2025).

The AI workshop provides convergent validity for these observations. Students in the follow-up cohort independently identified debugging as their greatest area of need for AI support (31.8% requesting additional practice), directly mirroring the experimental finding that debugging constituted the primary AI use case. This cross-cohort consistency suggests that the guideline addresses genuine pedagogical needs rather than experimental artifacts, and strengthens confidence in the practical relevance of structured AI guidance.

5.2 Implications for Educational Practice

Effective integration of generative AI into programming education requires more than technical access: pedagogical design is essential. Students need structured, explicit instruction on how to interact with AI tools in ways that support learning rather than merely facilitating task completion. The moderate guideline usage ($M = 2.83$) and perceived helpfulness ($M = 2.89$) point to implementation challenges: introducing the guideline immediately before task execution, under time pressure, likely limited deeper engagement. Future implementations should introduce the guideline gradually across sessions, embed it in the regular curriculum, provide practice with structured

feedback, and offer explicit meta-cognitive training on reflecting and regulating AI interactions.

The positive correlation between prior AI experience and perceived task usefulness ($r = 0.37$, $p = 0.035$) underscores the need for early, structured AI literacy development. Students who are introduced to AI tools through guided, pedagogically grounded frameworks appear better equipped to use these tools effectively. Workshop findings reinforce this: 63% of participants found the pacing appropriate, confirming the guideline can be delivered within typical instructional time frames, while the 76% who use AI chatbots frequently or daily highlight the urgency of embedding such guidance into curricula. Based on this evidence, AI awareness training has been integrated as a mandatory component of the introductory programming laboratory from the following semester onwards. This ensures that all students learn about the opportunities, challenges, and limitations of state-of-the-art AI models from semester one onwards.

Recent perspectives on AI in higher education emphasize that the challenge extends beyond individual tool use to a fundamental shift in learning culture (Kovanović et al., 2025). As Kovanović et al. argue, the focus must move from students producing content to engaging in reflective dialogue and critical evaluation — precisely the behaviours our guideline was designed to foster. This broader pedagogical context reinforces our finding that structured AI guidance develops meta-cognitive competencies that transcend immediate task performance.

5.3 Limitations and Future Research

Several limitations constrain the generalizability of these exploratory findings, which should be treated as hypothesis-generating rather than confirmatory. The sample ($n=36$) is a boundary condition of a single-course deployment; replication with larger, more diverse cohorts is necessary. The post-test relied exclusively on self-report questionnaire items, making results susceptible to demand characteristics and social desirability bias; future work should triangulate with objective measures such as a delayed debugging task or concept quiz. No formal rubric for correctness, completeness, or code quality was applied — a design limitation recommended for future replications. The specificity of the Snake game in C and the immediate guideline deployment under time pressure may further limit transfer and engagement depth.

Future research should investigate longitudinal and spaced-practice effects across multiple lab sessions, assess generalizability across languages and problem types, and determine optimal delivery for-

mats. Further work should examine individual differences in AI scaffolding benefit and develop continuous assessment models tracking student-AI interaction quality over time.

6 CONCLUSIONS

This article addresses the challenge of integrating generative AI effectively into programming education. Based on recent literature emphasizing structured support for novices, we developed a best-practice guideline fostering reflective AI use through prompt engineering, critical evaluation, and metacognitive awareness.

Our key contribution is showing that structured pedagogical guidance meaningfully influences how students interact with AI. Although the guideline did not significantly improve quantitative performance metrics, it demonstrably enhanced solution understanding and encouraged more sophisticated interaction patterns. For educators, this underscores that successful AI integration demands intentional instructional design, not merely tool access.

Supplementary deployment in an AI workshop (n=80) provided convergent validation: students independently identified debugging as their primary area of need, mirroring experimental findings and supporting generalizability across educational contexts. This led to AI literacy training becoming a mandatory component of first-semester programming education in future cohorts.

Future research should explore long-term effects, adaptability across diverse tasks, and how delivery formats influence engagement and adoption. As generative AI proliferates in education, evidence-based design is essential.

ACKNOWLEDGEMENTS

Use of AI Tools: During the preparation of this manuscript, the authors used Claude (Anthropic) to assist with data analysis and manuscript revision. The authors reviewed and edited all outputs and take full responsibility for the content. All experimental design, data collection, interpretation, and conclusions are solely the work of the authors.

REFERENCES

Duran, R., Sorva, J. and Leite, S. 2018 "Towards an Analysis of Program Complexity From a Cognitive Perspec-

tive," pp. 21-30.

Garrel, J. von and J. Mayer, "Artificial Intelligence in studies—use of ChatGPT and AI-based tools among students in Germany," in *Humanities and Social Sciences Communications*, vol. 10, 1, 10.1057/s41599-023-02304-7, 2023.

Grassini, S. 2023 "Shaping the Future of Education: Exploring the Potential and Consequences of AI and ChatGPT in Educational Settings," *Education Sciences*, vol. 13, no. 7, p. 692.

Huang, A. Y. Q. et al., "The impact of GenAI-enabled coding hints on students' programming performance and cognitive load in an SRL-based Python course," *British Journal of Educational Technology*, Apr. 2025.

Karlsruher Institute of Technology (KIT) "Using the opportunities of AI: Introduction and testing of bwGPT in Baden-Wuerttemberg." (Available online: <https://www.zml.kit.edu/english/bwGPT.php>)

Kovanović, V. et al., 2025 "Why universities need to radically rethink exams in the age of AI," *Nature*, vol. 648, pp. 35–37.

Lahtinen, E., Ala-Mutka, K. and H.-M. Järvinen, 2005. "A study of the difficulties of novice programmers," in *ACM SIGCSE Bulletin*, vol. 37, pp. 14-18.

Liu, R. et al., 2024 "Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education," in *Proc. 55th ACM Technical Symposium on Computer Science Education*, vol. 1, pp. 750-756.

Merrill, M. D., 2002. "First principles of instruction," *Educational Technology Research and Development*, vol. 50, no. 3, pp. 43-59.

Prather, J. et al., 2024. "The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers," in *Proc. 2024 ACM Conf. on International Computing Education Research*, vol. 1, pp. 469-486.

Salleh, S. M., Shukur, Z. and Judi, H. M., 2018. "Scaffolding Model for Efficient Programming Learning Based on Cognitive Load Theory," *International Journal of Pure and Applied Mathematics*.

Tian, H. et al., 2023 "Is ChatGPT the Ultimate Programming Assistant – How far is it?" arXiv:2304.11938 [cs].

Yilmaz, R. and Karaoglan Yilmaz, F. G., 2023 "Augmented intelligence in programming learning: Examining student views on the use of ChatGPT for programming learning," *Computers in Human Behavior: Artificial Humans*, vol. 1, no. 2, p. 100005.

Zhou, Z., Wang, S. and Qian, Y. 2021. "Learning From Errors: Exploring the Effectiveness of Enhanced Error Messages in Learning to Program," *Frontiers in Psychology*, vol. 12.